

HOSTEL MANAGEMENT SYSTEM

PROJECT REPORT

Name	ARYAN BARDIYA
Registration No.	25BAI10890
Course	JAVA Programming (CSE2006)
Slot	B22+B24
Project Type	Console-Based Core Java Application

A Core Java project demonstrating fundamental programming, object-oriented programming, exception handling, multithreading, and file handling concepts.

1. ABSTRACT

The Hostel Management System is a practical, console-based Java application developed to simplify the management of student and hostel room records. Hostel administration involves maintaining student details, allocating rooms, tracking occupied and available rooms, managing fee information, and updating or deleting records. Performing these activities manually can become repetitive and difficult to maintain as the number of students increases.

The proposed system provides a menu-driven environment where hostel records can be added, searched, updated, displayed, deleted, and stored for future use. It supports room allocation and vacation, hostel fee calculation, room availability checking, and a status-monitoring feature implemented using multithreading. Data persistence is achieved through simple text files stored inside a dedicated data folder.

The project applies a wide range of Core Java concepts, including classes and objects, constructors, encapsulation, inheritance, abstraction, interfaces, polymorphism, method overloading and overriding, enums, ArrayList, arrays, loops, conditions, exception handling, custom exceptions, multithreading, and file I/O.

2. INTRODUCTION

Hostel administration requires regular handling of student information and room-related activities. In a conventional setup, details may be maintained through registers or separate records, making it harder to quickly identify students, check room availability, update information, or maintain consistent records.

The Hostel Management System addresses these basic requirements through a simple console-driven application. Students can be registered with their academic and contact details, rooms can be allocated or vacated, and hostel fees can be calculated according to the number of months. The application also provides separate views for available and occupied rooms.

The system is implemented as a single-file Core Java project. Its purpose is not to replace a full-scale commercial hostel platform, but to demonstrate how fundamental Java programming concepts can be combined to create a useful real-world management application.

3. OBJECTIVES

- To build an interactive, menu-driven hostel management application.
- To maintain student information in an organized manner.
- To allocate and vacate hostel rooms efficiently.
- To display available and occupied rooms.
- To calculate hostel fees based on the number of months.
- To apply Classes, Objects, Constructors, and other OOP concepts in a practical scenario.
- To demonstrate inheritance, abstraction, interfaces, and polymorphism.
- To use ArrayList and arrays for managing application data.
- To validate user input and handle runtime errors using exception handling.
- To implement a custom exception for room-related errors.
- To demonstrate basic multithreading through a hostel status monitor.
- To store and retrieve records using Java file handling.

4. PROJECT SCOPE

The scope of this project is focused on basic hostel administration. It provides the functionality required to maintain student records, manage a fixed set of hostel rooms, calculate fees, and persist information locally without requiring an external database server. The application initializes 20 hostel rooms numbered from 101 to 120. Rooms are represented using the SINGLE, DOUBLE, and TRIPLE room types. Student records are stored dynamically using an ArrayList, while room information is maintained using an array.

Feature	Description & Purpose
Add Student	Registers a student using roll number, name, course, year, and phone number.
Display Students	Displays all registered student records with room and fee information.
Search Student	Finds a student using the roll number.
Allocate Room	Assigns an available room to a student after validating the student and room.
Vacate Room	Releases a student's allocated room and makes it available again.
Available Rooms	Displays rooms that are currently unoccupied.
Occupied Rooms	Displays rooms that are currently occupied.
Fee Calculation	Calculates hostel charges using the monthly fee and number of months.
Update Student	Updates name, course, year, phone, or fee status.
Delete Student	Removes a student and vacates the room first if required.
Save / Load Data	Stores and retrieves student, room, and fee records using text files.
Status Monitor	Runs a thread that displays total, occupied, and available rooms.

5. JAVA CONCEPTS USED

Java Concept	Role in Project Implementation
Classes & Objects	Classes such as Person, Student, Room, Hostel, and HostelStatusThread model the application's entities and operations.
Constructors	Default and parameterized constructors initialize Person, Student, Room, and Hostel objects.
Encapsulation	Important data members are private and accessed through getter and setter methods.
Inheritance	Student extends Person, allowing the student model to inherit common person functionality.
Abstraction	Person is an abstract class containing the abstract displayDetails() method.
Interface	FeeManagement defines calculateFee(int months), which is implemented by Student.
Polymorphism	A Person reference can refer to a Student object, demonstrating runtime method dispatch.
Method Overloading	calculateFee() and calculateFee(int months) provide different ways to calculate fees.
Method Overriding	Student overrides displayDetails() from Person.

Java Concept	Role in Project Implementation
Enum	RoomType defines SINGLE, DOUBLE, and TRIPLE room categories.
ArrayList	Stores student objects dynamically in the Hostel class.
Arrays	A Room[] array is used to maintain the 20 hostel rooms.
Loops & Conditions	for, for-each, while, if-else, and switch statements control application logic.
Exception Handling	try-catch-finally handles invalid numeric input and other runtime problems.
Custom Exception	RoomNotAvailableException handles invalid or unavailable room operations.
Multithreading	HostelStatusThread extends Thread and periodically displays hostel status.
File Handling	File, FileReader, FileWriter, BufferedReader, and BufferedWriter provide persistent text-file storage.
this / super	this refers to current object members; super invokes the parent constructor.
static / final	Static members maintain shared information; final constants define fixed fee and room limits.

6. SYSTEM DESIGN AND WORKING

The application starts by creating the data directory and loading previously saved records. It then displays the main menu continuously until the user chooses the Exit option. Each menu choice calls a dedicated method that performs the selected operation.

Student Management

Student records contain roll number, name, course, year, phone number, room number, room type, and fee status. The system prevents duplicate roll numbers and validates important numeric values before creating a Student object.

Room Management

The Hostel class maintains an array of 20 Room objects. During allocation, the application checks whether the student exists, the room exists, the room is unoccupied, and the student does not already have a room. A custom RoomNotAvailableException is raised when these conditions are not satisfied.

Fee Management

The monthly hostel fee is fixed at Rs. 5000. The Student class provides calculateFee() for one month and calculateFee(int months) for multiple months, demonstrating method overloading and interface implementation.

Status Monitoring

The HostelStatusThread displays the total number of rooms, occupied rooms, and available rooms five times, pausing for two seconds between displays. The main program starts the thread and waits for it using join().

7. PROJECT STRUCTURE

File / Directory	Description
HostelManagementSystem/	Root project directory.

File / Directory	Description
HostelManagementSystem.java	Single Java source file containing the main application and supporting classes.
data/	Directory created for persistent runtime data.
data/students.txt	Stores student records.
data/rooms.txt	Stores room numbers, occupancy status, and room types.
data/fees.txt	Stores fee-related information and status.

8. FILE HANDLING

The application uses lightweight text files instead of an external database. The data directory is created automatically if it does not exist. Student, room, and fee information is written to separate files and loaded again when required.

Student Data (data/students.txt)

Records are stored as comma-separated values containing roll number, name, course, year, phone, room number, room type, and fee status. Example:

```
101,Aryan Bardiya,Computer Science,1,9876543210,101,SINGLE,Paid
```

Room Data (data/rooms.txt)

Each room record stores the room number, occupancy status, and room type. Example:

```
101,Occupied,SINGLE
```

Fee Data (data/fees.txt)

Fee records contain the student's roll number, calculated monthly fee, and fee status. Example:

```
101,5000.0,Paid
```

When data is loaded, the program reads each file line by line, splits the stored values, reconstructs the corresponding objects, and synchronizes the student count with the loaded records. Data is also saved automatically before the application exits.

9. MAIN MENU

The application provides the following menu options:

Option	Operation
1	Add Student
2	Display All Students
3	Search Student
4	Allocate Room
5	Vacate Room
6	Display Available Rooms
7	Display Occupied Rooms

Option	Operation
8	Calculate Hostel Fee
9	Update Student Details
10	Delete Student
11	Save Data
12	Load Data
13	Start Hostel Status Monitor
14	Exit

10. TESTING

Test Case	Expected Result
Add valid student	Student is added successfully.
Add duplicate roll number	Duplicate student is rejected.
Invalid roll number/year	Appropriate validation error is displayed.
Search existing student	Student details are displayed.
Search unknown student	Student not found message is displayed.
Allocate available room	Room is allocated successfully.
Allocate occupied room	Room allocation error is displayed.
Allocate invalid room	Room does not exist error is displayed.
Allocate room to student already assigned	Operation is rejected.
Vacate allocated room	Room becomes available.
Vacate room for student without room	Error message is displayed.
Calculate hostel fee	Monthly and total fee are displayed.
Update student details	Selected information is updated.
Delete student with allocated room	Room is vacated and student is deleted.
Save data	Records are written to text files.
Load data	Previously saved records are restored.
Start status monitor	Thread displays hostel room statistics.
Exit	Data is saved before the program closes.

11. ADVANTAGES

- Simple and easy-to-use console-based interface.
- Centralizes basic student and room management operations.
- Provides quick room availability and occupancy information.
- Uses persistent local files without requiring a database server.
- Demonstrates a broad range of Core Java and OOP concepts in one application.

- Includes validation and exception handling to make operations safer.
- Easy to understand, test, modify, and extend for academic purposes.

12. LIMITATIONS

- The application is currently restricted to a console/terminal interface.
- Room management uses a fixed set of 20 rooms.
- Data is stored in text files rather than a relational database.
- The application does not provide network or multi-user access.
- There is no authentication or role-based admin system.
- The current room model tracks occupancy but does not provide detailed per-bed capacity management.

13. FUTURE ENHANCEMENTS

- Add a graphical user interface using JavaFX or Swing.
- Connect the application to MySQL or another relational database.
- Add administrator login and authentication.
- Support multiple hostel blocks and larger room inventories.
- Add check-in and check-out dates.
- Add online fee payment and automatic fee reminders.
- Add fine calculation for delayed payments.
- Provide advanced search and filtering.
- Generate student, room, and fee reports.
- Add email or notification functionality.

14. CONCLUSION

The Hostel Management System presents a practical application of Core Java concepts to a common administrative problem. The application combines student record management, room allocation, room vacancy tracking, fee calculation, local file persistence, exception handling, and multithreading within a single console-based system. The project demonstrates how theoretical Java concepts can be integrated into a functional application. Concepts such as inheritance, abstraction, interfaces, polymorphism, collections, arrays, custom exceptions, threads, and file I/O are used for specific purposes rather than being demonstrated in isolation. As a result, the project provides a clear academic example of applying Java programming fundamentals to a real-world management scenario.

15. AUTHOR

Name	ARYAN BARDIYA
Registration No.	25BAI10890
University	VIT Bhopal University
Course	JAVA Programming (CSE2006)
Slot	B22+B24